

Milestone 6 - 7 Phases Plan

D435i-style sensing, ROS 2 SLAM, Nav2 navigation, and KPI reporting for the MuJoCo + Unitree G1 maze project

Purpose

This document breaks the next project milestone into seven controlled phases so Codex can plan and execute one phase at a time. The goal is to avoid one giant integration task that mixes MuJoCo rendering, ROS 2 topics, TF, SLAM, Nav2, locomotion, and KPI reporting into a single hard-to-debug change.

Use this PDF as the source plan. For each phase, ask Codex to read or follow only the relevant phase, create a short file-by-file plan, then execute that phase only after you approve the plan.

Non-negotiable rules

- MuJoCo remains the simulator and physics source.
- RViz is used for ROS perception/navigation introspection, not as a replacement for MuJoCo.
- The physical realsense2_camera driver is not used in simulation. The MuJoCo bridge publishes RealSense-like topics instead.
- Ground truth may be logged for debug/evaluation, but final navigation must not secretly depend on MuJoCo ground truth pose.
- Every phase must produce visible artifacts under runs/visual or a clearly documented equivalent location.
- Do not rewrite working milestone/navigation/locomotion logic unless the change is required for the current phase.
- Do not move to the next phase until the current phase has a validation command, artifacts, README notes, and worklog notes.

Target architecture

```
MuJoCo G1 + simulated D435i
-> ROS 2 MuJoCo sensor bridge
-> /camera/depth/image_rect_raw
-> /camera/color/image_raw
-> /camera/*/camera_info
-> /imu/data
-> /tf, /tf_static, /joint_states, /clock
-> depthimage_to_laserscan
-> /scan
-> slam_toolbox
-> /map
-> Nav2
-> /cmd_vel
-> G1 walking-policy adapter
-> MuJoCo controls
```

Recommended Codex workflow

- Start each phase in Plan mode: ask for file-by-file plan, risks, validation commands, and artifacts.
- Approve or correct the plan.
- Execute only that phase.
- Run the phase validation command yourself.
- Commit or save the result before moving to the next phase.
- Use High reasoning for normal phases. Use Extra High only for ROS/TF/depth alignment debugging.

Template:

Create a concise implementation plan for Phase <N> from the Milestone 6 - 7 Phases Plan. Do not edit yet. Output: files to modify/create, validation commands, visible artifacts, risks, and definition of done. After I approve, execute Phase <N> only.

Phase 1: Add simulated D435i-style camera to the G1 in MuJoCo

Goal: Mount a visible D435i-style RGB-D + IMU sensor on the G1 and save camera-view artifacts before touching ROS.

Scope

- Add a D435i-style camera frame/body to the robot or world-building layer.
- Mount it on upper torso/chest/head area, facing forward and pitched slightly down about 5-15 degrees.
- Save RGB and depth images from the simulated camera.
- Include a visible camera marker/body if practical.
- Do not implement ROS, SLAM, Nav2, or /scan in this phase.

Expected artifacts

- runs/visual/d435i_mount_seed-123_final.png
- runs/visual/d435i_camera_view_seed-123_rgb.png
- runs/visual/d435i_camera_view_seed-123_depth.png
- runs/visual/d435i_mount_dashboard.html
- runs/visual/d435i_mount_summary.json

Validation commands

```
make d435i-visual-check SEED=123
make test
```

Definition of done

- A simulated D435i-style camera is visibly mounted on the robot.
- RGB and depth artifacts are saved and show whether the camera sees maze walls.
- Dashboard records camera pose, frame names, intrinsics/FOV, seed, and model/world.
- Existing project commands still work or any unrelated failure is documented.

Small Codex plan prompt for this phase

Plan Phase 1 only: add simulated D435i visual mount and RGB/depth artifacts in MuJoCo. Do not implement ROS bridge, SLAM, Nav2, or /scan. Provide files, validation commands, artifacts, risks, and then wait for approval.

Phase 2: Create ROS 2 MuJoCo sensor bridge

Goal: Publish RealSense-like ROS 2 Humble topics from the MuJoCo simulation.

Scope

- Create or extend `ros_ws/src/g1_mujoco_bridge` using Python ROS 2 nodes unless the repo strongly favors C++.
- Publish `/clock`, `/tf`, `/tf_static`, `/joint_states`, `/imu/data`, RGB image, depth image, and `CameraInfo` topics.
- Use `use_sim_time=true` and consistent `frame_id` values.
- Create a coherent TF tree: `map` -> `odom` -> `base_link` -> `torso` -> `d435i` frames where valid.
- If ground truth is needed for debug, publish it only as clearly labeled `/ground_truth` topics.

Expected artifacts

- `runs/visual/ros_bridge_seed-123_topics.txt`
- `runs/visual/ros_bridge_seed-123_topic_rates.txt`
- `runs/visual/ros_bridge_seed-123_tf_frames.pdf` or equivalent
- `runs/visual/ros_bridge_seed-123_rgb.png`
- `runs/visual/ros_bridge_seed-123_depth.png`
- `runs/visual/ros_bridge_seed-123_dashboard.html`
- `runs/visual/ros_bridge_seed-123_summary.json`

Validation commands

```
make ros-bridge-check SEED=123
ros2 topic list
ros2 topic hz /camera/depth/image_rect_raw
ros2 topic hz /imu/data
ros2 topic echo /camera/depth/camera_info --once
```

Definition of done

- ROS 2 sees the simulated D435i-like camera topics.
- RGB/depth/IMU topics publish at stable rates.
- `CameraInfo` is valid and matches image dimensions.
- TF tree is valid enough to inspect in RViz or tf2 tools.
- Bridge creates visible artifacts and dashboard.

Small Codex plan prompt for this phase

Plan Phase 2 only: create the ROS 2 MuJoCo sensor bridge for simulated D435i topics. Do not implement `/scan`, `slam_toolbox`, `Nav2`, or `KPI` yet. Provide files, launch/config structure, validation commands, artifacts, risks, and wait for approval.

Phase 3: Convert depth image to /scan

Goal: Use `depthimage_to_laserscan` to convert simulated D435i depth into a 2D LaserScan for future SLAM/Nav2.

Scope

- Create `d435i_depth_to_scan.yaml` or equivalent config.
- Input topics: `/camera/depth/image_rect_raw` and `/camera/depth/camera_info`.

- Output topic: /scan.
- Create launch file for bridge + depthimage_to_laserscan.
- Create RViz config showing TF and /scan; RobotModel and images if available.
- Do not implement slam_toolbox mapping or Nav2 in this phase.

Expected artifacts

- runs/visual/d435i_scan_seed-123_rviz.png
- runs/visual/d435i_scan_seed-123_rgb.png
- runs/visual/d435i_scan_seed-123_depth.png
- runs/visual/d435i_scan_seed-123_scan_overlay.png
- runs/visual/d435i_scan_seed-123_topic_rates.txt
- runs/visual/d435i_scan_seed-123_dashboard.html
- runs/visual/d435i_scan_seed-123_summary.json

Validation commands

```
make d435i-scan-check SEED=123
ros2 topic list | grep scan
ros2 topic hz /scan
ros2 topic echo /scan --once
```

Definition of done

- /camera/depth/image_rect_raw and /camera/depth/camera_info are published.
- /scan is published by depthimage_to_laserscan.
- /scan appears aligned with maze walls in RViz.
- Dashboard records scan angle/range configuration, valid ranges, topic rates, and TF status.

Small Codex plan prompt for this phase

Plan Phase 3 only: convert simulated D435i depth to /scan using depthimage_to_laserscan and validate in RViz.
Do not implement slam_toolbox, Nav2, or KPI yet. Provide files, launch/config details, validation commands, artifacts, risks, and wait for approval.

Phase 4: Build SLAM with slam_toolbox

Goal: Use /scan, TF, and odometry to create a recognizable occupancy map of the maze.

Scope

- Run the G1 through the maze with teleop or existing oracle/path-following control while mapping.
- Use /scan from Phase 3 and a valid odom -> base_link transform.
- Start with a fixed seed such as SEED=123.
- Save map outputs and compare against generated maze only for debug/evaluation.
- Do not add Nav2 autonomous navigation yet; the goal is mapping only.

Expected artifacts

- runs/visual/slam_seed-123_map.pgm
- runs/visual/slam_seed-123_map.yaml
- runs/visual/slam_seed-123_rviz.png
- runs/visual/slam_seed-123_dashboard.html
- runs/visual/slam_seed-123_tf_tree.pdf or equivalent
- runs/visual/slam_seed-123_bag or topic log if practical

Validation commands

```
make slam-map SEED=123
ros2 topic echo /map --once
ros2 topic hz /scan
ros2 run tf2_ros tf2_echo odom base_link
```

Definition of done

- slam_toolbox creates a recognizable occupancy map of the maze.
- Map, RViz screenshot, TF status, and dashboard are saved.
- Map coverage/time/scan rates are logged.
- Ground truth comparison, if used, is debug/evaluation only.

Small Codex plan prompt for this phase

Plan Phase 4 only: use /scan + TF + odom with slam_toolbox to build a map.
Do not implement Nav2 autonomous navigation yet. Provide config, launch, validation, artifacts, metrics, risks, and wait for approval.

Phase 5: Add Nav2 in SLAM mode

Goal: Run Nav2 while SLAM is active and verify that Nav2 produces /cmd_vel from goals and costmaps.

Scope

- Create g1_nav_bringup package if needed with launch and config files.
- Run slam_toolbox and Nav2 together with use_sim_time=true.
- Configure global/local costmaps, robot footprint, planner, controller, and velocity limits conservatively.
- Verify Nav2 sees obstacles and produces /cmd_vel.
- Do not yet require perfect G1 goal completion; first prove Nav2 pipeline and commands.

Expected artifacts

- runs/visual/nav2_slam_seed-123_rviz.png
- runs/visual/nav2_slam_seed-123_costmaps.png or screenshots
- runs/visual/nav2_slam_seed-123_path.svg or image
- runs/visual/nav2_slam_seed-123_cmd_vel.csv
- runs/visual/nav2_slam_seed-123_dashboard.html
- runs/visual/nav2_slam_seed-123_summary.json

Validation commands

```
make nav2-slam-demo SEED=123
ros2 topic echo /cmd_vel --once
ros2 topic list | grep costmap
ros2 lifecycle nodes
```

Definition of done

- Nav2 launches with SLAM active.
- A goal can be sent manually or by script.
- Nav2 publishes /cmd_vel.
- RViz shows map, costmaps, goal, and planned path.
- Dashboard records Nav2 state and limitations.

Small Codex plan prompt for this phase

Plan Phase 5 only: launch Nav2 in SLAM mode and verify Nav2 path/costmap and /cmd_vel output.

Do not optimize full maze completion yet and do not build KPI reporting. Provide files, configs, validation, artifacts, risks, and wait for approval.

Phase 6: Navigate to the maze goal

Goal: Connect Nav2 /cmd_vel to the G1 walking-policy adapter and navigate toward the maze goal without using ground truth for navigation.

Scope

- Implement or connect /cmd_vel -> G1 walking policy adapter.
- Clip velocities and yaw rates conservatively for humanoid stability.
- Support debug/oracle goal source and demo/autonomous goal source clearly labeled.
- Log commanded vs achieved velocity, contacts, recovery events, and status.
- Ground truth may be used for evaluation, not final navigation control.

Expected artifacts

- runs/visual/navigate_d435i_seed-123_rviz.png
- runs/visual/navigate_d435i_seed-123_mujoco_final.png
- runs/visual/navigate_d435i_seed-123_trajectory.svg
- runs/visual/navigate_d435i_seed-123_nav_path.svg
- runs/visual/navigate_d435i_seed-123_cmd_vel.csv
- runs/visual/navigate_d435i_seed-123_dashboard.html
- runs/visual/navigate_d435i_seed-123_summary.json

Validation commands

```
make navigate-d435i SEED=123
# expected summary fields:
# final_status, slam_enabled, nav2_enabled, sensor_source,
# ground_truth_used_for_navigation
```

Definition of done

- G1 attempts to navigate toward the goal using SLAM/Nav2 and /cmd_vel adapter.
- Summary clearly says whether ground truth was used for navigation; final answer should be false in demo mode.
- MuJoCo, RViz, trajectory, command timeline, and dashboard artifacts are saved.
- Failure modes are logged honestly if the robot times out or gets stuck.

Small Codex plan prompt for this phase

Plan Phase 6 only: connect Nav2 /cmd_vel to the G1 walking-policy adapter and navigate to the maze goal.
Keep ground truth debug/eval only. Provide files, configs, validation commands, artifacts, failure modes, and wait for approval.

Phase 7: KPI and reporting layer

Goal: Run multi-seed evaluations and generate a clear report/dashboard for assignment demo and defense.

Scope

- Create evaluation scripts for success rate, time to goal, path length, path efficiency, contacts, recoveries, clearance, map coverage, drift debug, and topic rates.
- Run multiple seeds and store summaries in CSV/JSON.
- Create HTML report with charts, key metrics, artifacts, and known limitations.

- Make metrics honest: separate navigation metrics from debug/evaluation metrics.
- Prepare demo-safe commands and a concise explanation of the architecture.

Expected artifacts

- runs/kpi/summary.csv
- runs/kpi/summary.json
- runs/kpi/report.html
- runs/kpi/success_by_seed.png
- runs/kpi/time_to_goal.png
- runs/kpi/path_efficiency.png
- runs/kpi/map_quality.png if evaluation-only comparison exists

Validation commands

```
make kpi-run SEEDS="1 2 3 4 5"
make kpi-report
```

Definition of done

- Multi-seed results are collected into summary CSV/JSON.
- Report includes success/failure per seed, time, path, contacts, map metrics, and topic rates.
- Demo dashboard shows camera/depth/scan/map/path/trajectory/status where possible.
- Known limitations and next actions are documented for interview defense.

Small Codex plan prompt for this phase

Plan Phase 7 only: create KPI collection and reporting for multi-seed SLAM/Nav2/G1 navigation runs.
Do not change navigation logic unless needed to log metrics. Provide files, validation commands, report artifacts, and wait for approval.

Cross-phase risks and debugging map

Symptom	Likely checks
Empty /scan	Check Phase 1 depth image, Phase 2 CameraInfo, topic remaps, and Phase 3 depthimage_to_laserscan config.
/scan rotated or offset	Check optical frame convention, static transforms, output_frame, and RViz fixed frame.
SLAM map drifts or twists	Check odom -> base_link, scan rate, timestamping, use_sim_time, and robot turning speed.
Nav2 path goes through walls	Check map, costmap inflation, robot footprint, obstacle layer, and TF alignment.
G1 collides despite good Nav2 path	Check /cmd_vel adapter, turn timing, velocity clipping, humanoid sway, and clearance margins.
Good terminal output but poor demo	Add or fix visible artifacts: MuJoCo render, RViz screenshot, dashboard, summary JSON, and trajectory SVG.

One-line rule for Codex

Plan all phases globally, but execute one phase at a time. Each phase must leave behind a visible artifact, a validation command, README notes, worklog notes, and a clear next action.